

Mega Man 2 em Assembly RISC-V: Desenvolvimento de um Jogo de Plataforma com Arquitetura Modular em Baixo Nível

Ana Caroline Freitas Brito¹, Arthur Santos Almeida², Felipe Da Graça Costa³,
Helio De Oliveira Dias⁴, Kamily Silva Andrade Crispim⁵,
Marcio Vinicius Da Silva Guimarães⁶, Vitor Ignacio Dos Santos Valcarcel⁷

Resumo— Este artigo apresenta o desenvolvimento de uma releitura do clássico jogo Mega Man 2, implementada inteiramente em linguagem Assembly RISC-V (ISA RV32IM). O projeto foi concebido como requisito avaliativo da disciplina de Organização e Arquitetura de Computadores (OAC) da Universidade de Brasília, sendo executável tanto no simulador FPGRARS quanto na placa FPGA DE1-SoC. A arquitetura de software adota uma organização modular em três camadas — Game, Engine e Entidades — totalizando mais de 6650 linhas de código-fonte escritas por sete desenvolvedores. O sistema implementa: motor de renderização com *tile mapping* e *frustum culling* bidimensional; motor de física com gravidade escalar, colisão AABB contra o mapa de tiles e resolução por alinhamento; sistema de câmera com *scrolling* horizontal e vertical; reprodução musical polifônica MIDI não bloqueante com efeitos sonoros sequenciais; máquina de estados finitos (FSM) de oito estados para o jogador; três tipos de inimigos com inteligências artificiais distintas; sistema de habilidades alternáveis (Freeze e High Jump) com consumo de energia; HUD com barras de vida e energia segmentadas; e transição dinâmica entre duas áreas temáticas distintas por detecção de tiles de porta. Os resultados demonstram a viabilidade de construir um jogo de plataforma complexo sob as restrições severas do Assembly, cumprindo integralmente os dez requisitos estipulados pela disciplina.

Palavras-chave— Assembly RISC-V, Mega Man 2, FPGRARS, FPGA DE1-SoC, Arquitetura de Computadores, Máquina de Estados Finitos, *Tile Mapping*, MMIO.

I. INTRODUÇÃO

O desenvolvimento de jogos em linguagem Assembly constitui um desafio acadêmico que expõe os fundamentos da arquitetura de computadores de forma concreta. Ao abdicar das abstrações fornecidas por linguagens de alto nível e *game engines* comerciais, o programador é compelido a gerenciar diretamente registradores, endereçamento de memória, controle de fluxo e periféricos de entrada/saída [1].

Este artigo documenta o desenvolvimento de uma releitura computacional do jogo Mega Man 2 — originalmente lançado pela Capcom para o Nintendo Entertainment System (NES) em 1988 [2] — programada nativamente em Assembly RISC-V. O projeto foi elaborado como requisito avaliativo da disciplina de Organização e Arquitetura de Computadores (OAC) da Universidade de Brasília (UnB), sob orientação do Prof. Marcus Vinicius Lamar, utilizando a ISA RV32IM (instruções inteiras com extensão de multiplicação).

O escopo do projeto compreende dez requisitos técnicos obrigatórios: (1) música e efeitos sonoros; (2) ataque base do jogador; (3) movimentação e animação; (4) duas habilidades especiais alternáveis; (5) HUD com vida e energia; (6) itens coletáveis; (7) duas áreas temáticas distintas com porta de transição; (8) três tipos de inimigos com IAs distintas, incluindo um chefe; (9) *background* móvel com *scrolling*; e (10) este artigo científico.

O sistema foi projetado para execução em dois ambientes: o simulador FPGRARS [6] e a placa FPGA Altera DE1-SoC, utilizando o processador RISC-V v24 com saída VGA e teclado PS/2.

O presente documento está segmentado como segue: a Seção II apresenta a fundamentação teórica e trabalhos relacionados; a Seção III detalha a metodologia de implementação e arquitetura de software; a Seção IV apresenta os resultados obtidos e desafios superados; a Seção V conclui o estudo com proposições para trabalhos futuros.

II. FUNDAMENTAÇÃO TEÓRICA E TRABALHOS RELACIONADOS

A. A Arquitetura RISC-V

O RISC-V é uma ISA aberta desenvolvida na Universidade da Califórnia, Berkeley, seguindo o paradigma *Reduced Instruction Set Computer* [1]. Diferentemente de arquiteturas CISC como x86, o RISC-V adota estritamente o modelo

Trabalho da disciplina de Organização e Arquitetura de Computadores (OAC), sob orientação do Prof. Marcus Vinicius Lamar.

¹242001526, Universidade de Brasília (UnB), 242001526@aluno.unb.br

²242009990, Universidade de Brasília (UnB), 242009990@aluno.unb.br

³252009161, Universidade de Brasília (UnB), 252009161@aluno.unb.br

⁴232009520, Universidade de Brasília (UnB), 232009520@aluno.unb.br

⁵242009945, Universidade de Brasília (UnB), 242009945@aluno.unb.br

⁶242001553, Universidade de Brasília (UnB), 242001553@aluno.unb.br

⁷242010006, Universidade de Brasília (UnB), 242010006@aluno.unb.br

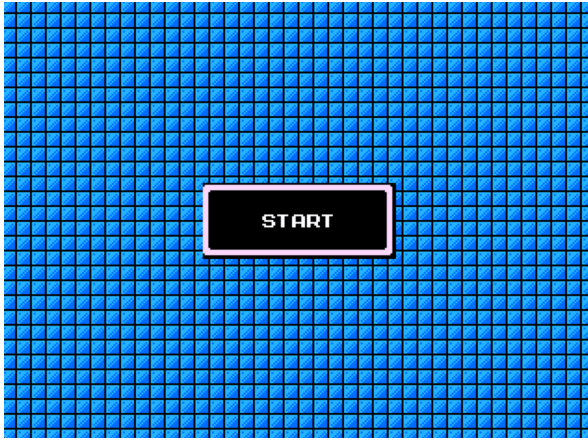


Fig. 1. Tela de Start do Mega Man 2 em RISC-V.

load/store: operações aritméticas e lógicas operam exclusivamente sobre o banco de 32 registradores de 32 bits ($x0-x31$), e o acesso à memória é restrito às instruções *lw/sw* (e suas variantes *lh, lb, lhu, lbu*).

Este paradigma impõe ao desenvolvedor uma gestão cuidadosa dos registradores: argumentos são passados em *a0-a7*, valores salvos em *s0-s11*, temporários em *t0-t6*, e o endereço de retorno em *ra*. A instrução *jal* (Jump and Link) sobrescreve *ra*, exigindo empilhamento rigoroso via ponteiro de pilha (*sp*) em chamadas aninhadas.

B. Memory-Mapped I/O e Framebuffer

Em ambientes sem sistema operacional, a comunicação com periféricos dá-se por *Memory-Mapped I/O* (MMIO). No FPGRARS e na DE1-SoC, o *framebuffer* ocupa regiões contíguas de memória. O endereço de um pixel na tela é calculado pela equação:

$$\text{Addr} = \text{Base} + (Y \times W + X) \quad (1)$$

onde *Base* é o endereço inicial do *framebuffer* (ex.: $0xFF000000$), $W = 320$ é a largura da tela, e cada pixel ocupa 1 byte no formato de paleta de 256 cores. O FPGRARS utiliza *double buffering*: dois *framebuffers* em $0xFF000000$ e $0xFF100000$ são alternados via registro de controle em $0xFF200604$, eliminando o *flickering* visual.

C. Ferramentas e Trabalhos Anteriores

O ecossistema de desenvolvimento se beneficiou de ferramentas da comunidade acadêmica da UnB. O FPGRARS [6], criado por Leonardo Riether, é um simulador RISC-V com display gráfico e suporte a MIDI, executando até 200 vezes mais rápido que o RARS tradicional. O conversor *png2oac* [5] foi utilizado para transformar sprites do Mega Man 2 extraídos de comunidades de preservação [3] em arrays binários compatíveis com a diretiva *.data*. Para a trilha sonora, arquivos MIDI do jogo original [4] foram convertidos para arrays de eventos reproduzíveis pelas *ecalls* de áudio do simulador, seguindo a metodologia do conversor MIDI-RISC-V [7]. Projetos de semestres anteriores de OAC, como

Metroid e Celeste em RISC-V, serviram como referência arquitetural para a organização modular do código.

III. METODOLOGIA E ARQUITETURA DE SOFTWARE

A. Arquitetura em Três Camadas

O projeto adota uma arquitetura modular em três camadas, projetada para suportar o desenvolvimento colaborativo de sete programadores com controle de versão Git:

- **Game** (*main.s*): orquestra o *game loop*, gerencia estados globais (Título, Jogando, Game Over), carrega mapas e coordena atualizações.
- **Engine** (*engine/*): subsistemas reutilizáveis e independentes de entidades — renderização, física, câmera, input, animação, música e efeitos sonoros.
- **Entidades** (*entities/*): código comportamental dos atores — jogador, inimigos tipo 1 e 2, chefe e itens coletáveis.

Cada módulo expõe rotinas de interface pública com prefixo padronizado (ex.: *PLAYER_SETUP*, *ENEMY1_UPDATE*, *RENDER_MAPA*), seguindo convenções de nomenclatura documentadas. O *game loop* principal executa a sequência:

```
1 GAME_LOOP:
2     call READ_INPUT
3     call GAME_UPDATE_STATE
4     call SFX_UPDATE
5     call GAME_RENDER_STATE
6     call PRESENT_FRAME
7     call SWAP_FRAMEBUFFER
8     call WAIT_FRAME
9     j     GAME_LOOP
```

Listing 1. Game Loop principal em *main.s*.

B. Motor de Renderização e Câmera (Req. 9)

O sistema de renderização é baseado em *tile mapping*: cada mapa é uma matriz de 40×30 tiles de 192×160 pixels, onde cada byte referencia um índice no *tileset* compartilhado. O *tileset* de 192×160 pixels é carregado integralmente na seção *.data*, e uma tabela de offsets pré-calculados (*MAPA_TILESET_OFFSETS*) mapeia cada índice de *tile* ao seu deslocamento em bytes no *tileset*, evitando multiplicações em tempo de execução.

A câmera centraliza o jogador na tela subtraindo 160 pixels (metade da largura) da posição X e 120 pixels da posição Y, com clamping nos limites do mapa. A posição da câmera é alinhada a múltiplos de 4 pixels via operação *and* com máscara -4 , eliminando artefatos visuais de subpixel:

```
1     addi t1, t1, -160    # Centraliza X
2     li   t4, -4
3     and  t1, t1, t4      # Alinha a multiplo de 4
4     sh   t1, 0(t3)      # Salva em BG_POS
```

Listing 2. Alinhamento da câmera em *camera.s*.

A otimização principal de renderização é o *frustum culling* bidimensional: em vez de iterar sobre todos os $40 \times 30 = 1200$ tiles do mapa, o renderizador calcula a coluna e linha iniciais visíveis dividindo a posição da câmera pela dimensão do *tile* (via *srl* por 4 bits), e itera apenas sobre as

$21 \times 16 = 336$ tiles visíveis na tela de 320×240 pixels. Adicionalmente, quando um tile está completamente dentro da tela e alinhado em memória, uma rotina *fast path* copia 4 palavras de 32 bits por linha (16 bytes = largura do tile), reduzindo o número de instruções de store por um fator de 4.

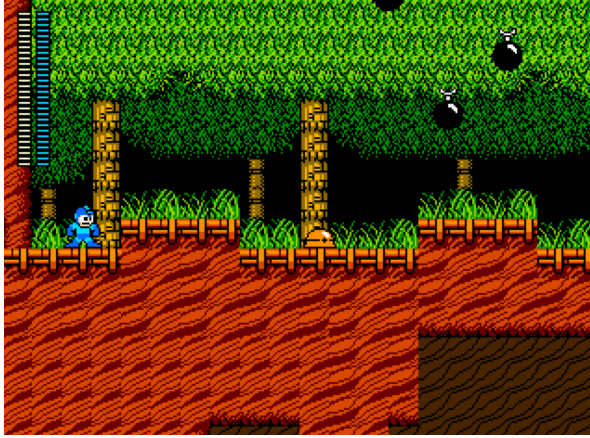


Fig. 2. Gameplay na Área 1 (Wood Man Stage) com *scrolling* ativo, HUD de vida e energia visíveis, inimigo tipo 1 (Met) e inimigo tipo 2 (Batton).

C. Motor de Física e Colisão (Req. 3)

O motor de física implementa gravidade escalar e resolução de colisão contra o mapa de tiles. A camada de colisão é uma matriz paralela à visual, onde cada byte indica o tipo de tile: 0 (vazio), 1 (sólido), 2 (escada), 3 (porta).

A gravidade é aplicada incrementando a velocidade vertical a cada frame, com velocidade terminal limitada a 6 pixels/frame:

```
1 _PHYSICS_RESOLVE_VERTICAL_GRAVITY:
2     addi s5, s5, 1      # vel_y += 1
3     li    t0, 6         # velocidade terminal
4     ble   s5, t0, _DONE
5     mv    s5, t0        # clamp
```

Listing 3. Gravidade com velocidade terminal em physics.s.

A resolução de colisão horizontal e vertical utiliza o método de *tile-aligned correction*: ao detectar uma sobreposição com um tile sólido, a posição da entidade é recuada para o limite do tile mais próximo usando operações de *srli/slli* (divisão e multiplicação por 16 via shifts). São testados dois pontos de prova por direção (meio e borda do hitbox), garantindo detecção robusta para entidades maiores que um tile.

O sistema de escadas merece destaque: tiles de escada funcionam como chão sólido quando o jogador está por cima (permitindo caminhar sobre elas), mas quando o jogador pressiona cima/baixo, o flag `PLAYER_IS_ON_LADDER` é ativado, desabilitando a gravidade e permitindo movimento vertical livre. Um flag de `PLAYER_LADDER_RELEASED` impede re-agarre imediato ao soltar a escada com pulo.

D. Máquina de Estados do Jogador (Req. 2, 3, 4)

O jogador implementa uma FSM com oito estados distintos, gerenciados pela rotina `PLAYER_UPDATE_STATE`:

Idle (0), Andando (1), No Ar (2), Atirando (3), Atirando no Ar (4), Na Escada (5), Atirando na Escada (6) e Knockback (7). Cada estado determina o sprite exibido e as ações permitidas.

O **ataque base** (Req. 2) suporta até 3 projéteis simultâneos gerenciados em arrays paralelos (`PLAYER_SHOTS_ACTIVE`, `PLAYER_SHOTS_X/Y`, `PLAYER_SHOTS_DIRECTION`). Ao pressionar o botão de tiro (J), a rotina `PLAYER_START_SHOOT` busca o primeiro slot livre, posiciona o projétil relativo ao jogador respeitando a direção, e dispara o efeito sonoro `SFX_BUSTER` via *tail call* (otimização que evita empilhamento adicional usando `j` em vez de `call`). Os projéteis se movem a 8 pixels/frame e são desativados ao sair dos limites da tela ou ao colidir com inimigos.

As **habilidades especiais** (Req. 4) são alternadas pelo botão L, ciclando entre três modos de arma (`PLAYER_WEAPON_MODE`):

- 1) **Normal (Buster)**: tiro padrão descrito acima.
- 2) **Freeze (Time Stopper)**: consome 4 pontos de energia (MP) e ativa `PLAYER_FREEZE_TIMER` por 90 frames, paralisando todos os inimigos. Durante o *freeze*, as rotinas `ENEMY1_UPDATE`, `ENEMY2_UPDATE` e `BOSS_UPDATE` não são chamadas.
- 3) **High Jump**: consome 2 MP por pulo e aplica velocidade inicial de -14 pixels/frame (contra -8 do pulo normal), permitindo alcançar plataformas superiores inacessíveis.

A troca de habilidade altera também a paleta visual do Mega Man: a rotina `RENDER_ENTITY` verifica `RENDER_COLOR_ADD` e aplica uma soma aritmética aos pixels azuis do sprite (cor 216 e 240), simulando uma troca de paleta sem duplicar assets.

E. Inteligências Artificiais dos Inimigos (Req. 8)

O jogo implementa três tipos de inimigos com IAs distintas, cada um com 5 instâncias por mapa (exceto o Boss, exclusivo do Mapa 2).

Inimigo Tipo 1 — Met (Patrulheiro com Escudo): Implementado em `enemy1.s`, opera em dois estados: **HIDED** (dormindo sob o escudo) e **PATROLLING** (ativo). No estado **HIDED**, o inimigo é *invulnerável* — tiros do jogador *ricocheteiam*, revertendo a direção do projétil (`xori t3, t3, 1`) com acréscimo de velocidade vertical negativa e efeito sonoro `SFX_DINK`. O inimigo ativa-se quando o jogador entra num raio de 80 pixels de distância horizontal, transitando para **PATROLLING** com direção inicial apontada para o jogador. Neste estado, patrulha a 1 pixel/frame, inverte ao colidir com tiles sólidos, e dispara rajadas triplas (projéteis reto, diagonal +3 e diagonal -3) a cada 80 frames. Retorna ao **HIDED** quando o jogador se afasta.

Inimigo Tipo 2 — Batton (Perseguição Aérea): Implementado em `enemy2.s`, é um inimigo voador com IA de perseguição bidirecional. Ao detectar o jogador dentro de 80 pixels, persegue-o movendo-se a 1 pixel/frame em ambos os eixos X e Y simultaneamente, convergindo para a posição do jogador. Crucialmente, *ignora a colisão com o terreno* (não

chama `PHYSICS_RESOLVE`), simulando voo que atravessa paredes. Não dispara projéteis — causa dano exclusivamente por contato corporal (4 HP). Possui animação de voo de 4 frames.

Boss — Heat Man (FSM de 7 Estados): Implementado em `boss.s` com 40 HP e FSM de sete estados: `INACTIVE` (dormindo até proximidade), `INTRO` (60 frames de animação de entrada), `IDLE` (24 frames de pausa), `THROW` (dispara 3 rajadas triplas nos frames 36, 24 e 12), `DASH` (avança a 4 px/frame por 22 frames em direção ao jogador), `JUMP` (pulo com $v_y = -7$ e deslocamento horizontal a 1 px/frame por 72 frames) e `DEAD`. O ciclo de ataques segue um padrão determinístico `THROW → DASH → JUMP`, gerenciado por um contador modular. Cada rajada tripla dispara 3 projéteis simultâneos: horizontal ($v_y = 0$), diagonal ascendente ($v_y = -2$) e diagonal descendente ($v_y = +2$). O pool de 12 projéteis opera em arrays paralelos independentes. Ao ser derrotado (HP = 0), uma animação de explosão é disparada e um timer de 120 frames é iniciado antes do retorno à tela de título.

F. Áreas Temáticas e Transição (Req. 7)

O jogo possui dois mapas de 640×480 pixels (40×30 tiles): Wood Man Stage (Mapa 1) e Heat Man Stage (Mapa 2), cada um com tileset visual, camada de colisão e posições de entidades distintas. Os mapas são gerenciados por descritores de 36 bytes contendo ponteiros para todas as tabelas relevantes. A transição ocorre quando o centro do hitbox do jogador coincide com um tile de tipo 3 (porta) no Mapa 1, detectado pela rotina `GAME_CHECK_MAP_TRANSITION`:

```
1  call PHYSICS_GET_COLLISION_TILE
2  call PHYSICS_IS_DOOR_TILE
3  beqz a0, _FALSE
4  la    a0, MAPA2_DESCRIPTOR
5  call GAME_LOAD_MAP
```

Listing 4. Detecção de transição de mapa em `main.s`.

A rotina `GAME_LOAD_MAP` reseta a câmera, reposiciona todas as entidades usando as tabelas do novo mapa, reinicializa inimigos e itens, e troca a música de fundo (Wood Man → Heat Man).

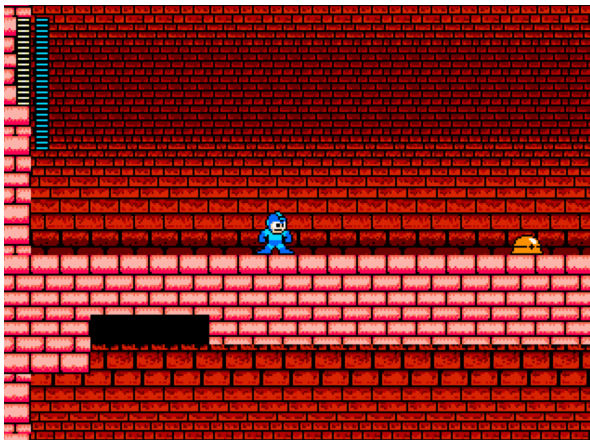


Fig. 3. Gameplay na Área 2 (Heat Man Stage) após a transição pela porta. Note o tileset visual distinto.

G. HUD — Heads-up Display (Req. 5)

O HUD é renderizado por `hud.s` como última camada sobre o `framebuffer`, garantindo que não seja ocluído pelo cenário. A barra de vida exibe um sprite de 3 pixels de altura (`HUD_SPRITE_LIFEBAR`) empilhado verticalmente por cada ponto de HP do jogador (máximo 28 segmentos), posicionada em (8,8) pixels. A barra de energia é renderizada ao lado ($x = 18$), mas ao invés de chamar `PRINT`, a rotina `HUD_RENDER_ENERGY` recolora manualmente os pixels: percorre cada byte do sprite e substitui pixels não-pretos pela cor 240 (azul energia), utilizando operações byte a byte (`lbu/sb`).

H. Itens Coletáveis (Req. 6)

Implementados em `items.s` (333 linhas), os itens são gerados *dinamicamente* por inimigos derrotados, utilizando um Gerador de Números Pseudoaleatórios (PRNG) do tipo Linear Congruential Generator (LCG) com os coeficientes clássicos da `glibc`: $seed = seed \times 1103515245 + 12345$, semente inicial `0x12345678`. Ao morrer, cada inimigo chama `ITEMS_TRY_SPAWN`: com 30% de chance ($valor < 77$ de 256) gera uma cápsula de HP; caso contrário, outra chance de 30% gera uma de MP, resultando em $\approx 51\%$ de probabilidade total de *drop*. Os itens gerados possuem gravidade própria (aceleração de 1 px/frame², velocidade terminal de 5 px/frame), caindo até pousar em plataformas via `PHYSICS_RESOLVE_VERTICAL`. O pool comporta até 8 itens simultâneos em arrays de 20 bytes cada. A coleta ocorre por teste AABB, restaurando 4 HP ou MP (limitado ao máximo de 28). Cápsulas possuem animação de 2 frames alternados.

I. Sistema de Áudio (Req. 1)

A **música** é reproduzida por `music.s` de forma não bloqueante. Os dados musicais são arrays com cabeçalho (contagem de eventos e pausa final) seguidos de eventos de 8 bytes (delay, duração, nota MIDI, instrumento, volume). A rotina `MUSIC_UPDATE`, chamada a cada frame, compara o tempo do sistema (`ecall 30`) com o timestamp do próximo evento agendado. Quando o tempo é atingido, a nota é disparada via `ecall 31` (MIDI *note-on*) e o índice avança. Ao fim da sequência, o índice retorna a 0 e a pausa final é adicionada, criando um loop contínuo. Duas trilhas são utilizadas: Wood Man Stage e Heat Man Stage, correspondendo aos mapas.

Os **efeitos sonoros** (SFX) operam em `sfx.s` com sistema independente. Cada SFX é uma sequência de trios (nota, duração, instrumento): `SFX_BUSTER` (5 notas curtas ascendentes), `SFX_ENEMY_HIT` (9 notas de impacto), `SFX_PLAYER_HIT` (8 notas), `SFX_TIME_STOPPER` (10 notas em instrumento 4), e `SFX_DINK` (nota única de marimba para rebatida). A rotina `SFX_UPDATE` avança para a próxima nota quando a duração da anterior expira, medida por `ecall 30`.

J. Sistema de Dano, Knockback e Input Multi-Plataforma

Quando o jogador colide com um inimigo ou projétil inimigo, a rotina `PLAYER_CHECK_DAMAGE_COLLISION`

aplica o dano correspondente (4 HP por contato corporal, 2 HP por tiro) e ativa o estado de *knockback*: o jogador é empurrado na direção oposta ao inimigo com velocidade de 4 pixels/frame por 360 ms, durante os quais o controle do jogador é travado (PLAYER_STATE_KNOCKBACK). Simultaneamente, um timer de invulnerabilidade de 1800 ms é ativado, durante o qual o sprite do jogador pisca (alternando visibilidade a cada 4 frames via `srli t1, t1, 2; andi t1, t1, 1`).

O sistema de **input** (`input.s`) implementa abstração multi-plataforma: lê o *keymap* de 128 bits do FPGRARS em `0xFF200520` e mapeia teclas físicas para 8 flags lógicas (LEFT, RIGHT, UP, DOWN, SHOOT, JUMP, SWITCH, START). Três mapas de scancodes são suportados — macOS, Linux, Windows e FPGA PS/2 — com auto-deteção no primeiro frame via teste de padrões de bits. A cada frame, são computados `INPUT_CURRENT` (teclas seguradas), `INPUT_PRESSED` ($current \wedge \neg previous$, recém-pressionadas) e `INPUT_RELEASED` ($previous \wedge \neg current$), permitindo detecção precisa de borda para ações como pulo e tiro.

K. Compatibilidade com FPGA DE1-SoC

O arquivo `fpga.s` constitui o ponto de entrada alternativo para a placa DE1-SoC. Ele inclui `MACROSv24.s`, que instala o tratador de exceções do processador RISC-V v24, e `SYSTEMv24.s`, que implementa as *ecalls* usadas pelo jogo. O script `fpga/build.sh` gera os arquivos `del_text.mif` e `del_data.mif` para programação via Quartus. Os controles mapeiam para teclado PS/2 (WASD + JKL), e a saída de vídeo utiliza o barramento VGA da placa.

IV. RESULTADOS OBTIDOS E DESAFIOS

A. Métricas do Projeto

O projeto totaliza aproximadamente 6.500 linhas de código Assembly escritas à mão, distribuídas em 17 arquivos-fonte, além de 14 arquivos de dados de mapas/entidades, 12 assets de sprites e ≈ 300 KB de dados binários. A Tabela I resume as métricas principais.

TABLE I
MÉTRICAS DO PROJETO

Métrica	textbf{Valor}
Linhas de código (Assembly)	≈ 6650
Dados de assets (.mif)	≈ 86 KB
Arquivos-fonte	17
Estados da FSM do jogador	8
Estados da FSM do Boss	7
Tipos de inimigos	3
Instâncias por mapa	11-12
Dimensão dos mapas	640×480 px
Resolução da tela	320×240 px
Tiles renderizados por frame	≤ 336
Projéteis simultâneos	3
Trilhas musicais	2
Efeitos sonoros distintos	6

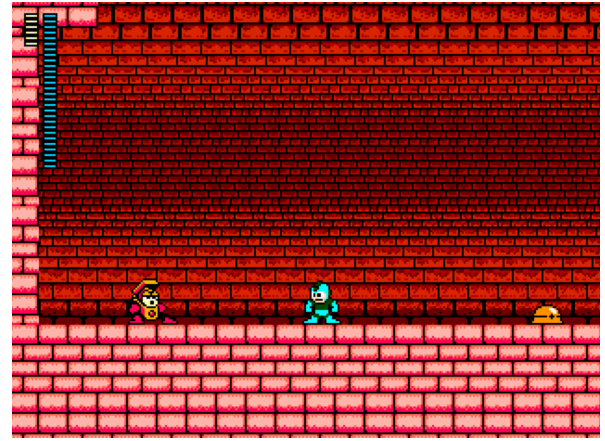


Fig. 4. Batalha contra o Boss (Heat Man) na Área 2. O sistema de deflexão de projéteis e I-frames exige troca tática de habilidades pelo jogador.

B. Conformidade com os Requisitos

Todos os dez requisitos estipulados pela disciplina foram atendidos integralmente:

- 1) **Música e SFX:** Duas trilhas MIDI polifônicas não bloqueantes com 6 efeitos sonoros sequenciais independentes.
- 2) **Ataque base:** Mega Buster com 3 projéteis simultâneos a 8 px/frame.
- 3) **Movimentação e animação:** FSM de 8 estados com sprites distintos, flip horizontal e piscar de invulnerabilidade.
- 4) **Habilidades:** Freeze (paralisa inimigos por 90 frames) e High Jump (pulo de velocidade -14 vs -8).
- 5) **HUD:** Barras de vida (28 segmentos, vermelha) e energia (28 segmentos, azul) renderizadas sobre o cenário.
- 6) **Itens:** Cápsulas de HP e MP com animação de 2 frames, coletáveis por colisão AABB.
- 7) **Áreas distintas:** Wood Man Stage e Heat Man Stage com tilesets, músicas e entidades diferentes, separadas por porta.
- 8) **Inimigos:** Met (patrulha horizontal), Batton (perseguição voadora) e Heat Man (Boss com FSM, I-frames e deflexão).
- 9) **Scrolling:** Câmera horizontal e vertical com *frustum culling* e alinhamento a 4 pixels.
- 10) **Documentação:** Esse artigo IEEE.

C. Desafios Técnicos Superados

1) **Otimização de Renderização por Frustum Culling:** A primeira implementação iterava sobre todos os 1200 tiles do mapa a cada frame, resultando em taxa de quadros inaceitável. A solução de *frustum culling* reduziu a iteração para no máximo 336 tiles (os visíveis na tela), e a rotina *fast path* com cópias de 4 bytes (`lw/sw` em vez de `lb/sb`) reduziu o número de instruções por tile em 75%, restaurando a fluidez a 30+ FPS.

2) **Vazamento de Pilha (Memory Leak de Stack):** O maior desafio de integração envolveu vazamentos na pilha de execução. Com 7 desenvolvedores alocando e desalocando

sp de forma inconsistente (ex.: alocar 20 bytes mas desalocar 16), o ra era corrompido, causando saltos para endereços inválidos. A solução foi adotar um cabeçalho de contrato obrigatório em cada sub-rotina:

```
1 # ROTINA_NOME
2 # Uso da Pilha (20 bytes):
3 # 0(sp) -> ra
4 # 4(sp) -> s1
5 # 8(sp) -> s2 (descricao)
```

Listing 5. Padrão de contrato de pilha adotado.

3) *Sistema de Escadas*: A interação entre escadas e gravidade exigiu lógica elaborada: tiles de escada devem atuar como chão sólido quando o jogador caminha por cima, mas permitir escalada quando cima/baixo são pressionados. A solução implementa dois testes distintos (pés dentro da escada vs. escada logo abaixo dos pés) e um flag de LADDER_RELEASED para evitar re-agarre involuntário após pulo.

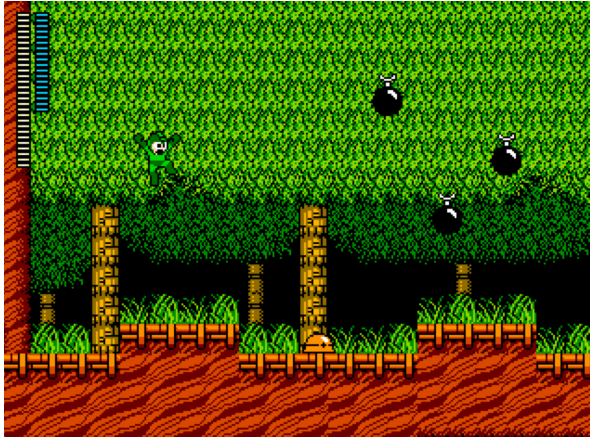


Fig. 5. Mega Man Pulando com o Super Jump. Mas ainda, realização da sua troca de cor indicando a utilização da habilidade.

V. CONCLUSÕES E TRABALHOS FUTUROS

O desenvolvimento de uma releitura de Mega Man 2 inteiramente em Assembly RISC-V demonstrou que é viável construir um jogo de plataforma complexo sob as restrições severas de uma linguagem de baixo nível. O projeto forçou a equipe a compreender profundamente: a gestão de registradores e pilha; a aritmética de endereçamento para framebuffer e MMIO; técnicas de otimização como *frustum culling* e cópias alinhadas; e a coordenação modular de código em equipe.

A arquitetura em três camadas provou-se essencial para a produtividade: isolando a Engine (render, física, câmera) das Entidades (player, inimigos, itens), foi possível depurar problemas de colisão sem afetar a lógica de IA, e vice-versa.

Como **trabalhos futuros**, propõe-se: (i) implementação de compressão Run-Length Encoding (RLE) para sprites, reduzindo o consumo de memória da seção .data; (ii) ampliar a integração já funcional com a DE1-SoC, incorporando seus periféricos físicos adicionais: switches para seleção de habilidades, displays de sete segmentos para informações do HUD e LEDs para feedback de dano; (iii) adição de mais fases com carregamento dinâmico de mapas por arquivo; e

(iv) implementação de um sistema de pontuação e tela de vitória ao derrotar o Boss. O jogo foi validado fisicamente na placa FPGA DE1-SoC, com saída VGA, entrada por teclado PS/2 e reprodução de música e efeitos sonoros.

AGRADECIMENTOS

Os autores agradecem ao Prof. Marcus Vinicius Lamar pela orientação na disciplina de OAC, à equipe de monitores pelo suporte nas sessões de depuração, aos criadores das ferramentas comunitárias FPGRARS, png2oac e conversor MIDI-RISC-V, e à Universidade de Brasília por viabilizar este projeto prático.

REFERENCES

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2017.
- [2] Capcom, *Mega Man 2* [Software de Jogo], Nintendo Entertainment System (NES), 1988.
- [3] The Spriters Resource. “Mega Man 2 Sprites”. Disponível em: <https://www.sprisers-resource.com/nnes/mm2/>. Acesso em: Jul. 2026.
- [4] KHInsider. “Mega Man 2 NES Music”. Disponível em: <https://www.khinsider.com/midi/nnes/mega-man-2>. Acesso em: Jul. 2026.
- [5] ABMHub. “Repositório png2oac”. Disponível em: <https://github.com/ABMHub/png2oac>. Acesso em: Jul. 2026.
- [6] L. Riether. “FPGRARS — Simulador RISC-V com display gráfico”. Disponível em: <https://github.com/LeoRiether/FPGRARS>. Acesso em: Jul. 2026.
- [7] L. Megiorin. “Midi2RiscV — Conversor MIDI para RISC-V”. Disponível em: <https://github.com/Luke0133/Midi2RiscV>. Acesso em: Jul. 2026.
- [8] V. Lisboa. “LAMAR — Learning Assembly for Machine Architecture and RISC-V”. Disponível em: <https://github.com/victorlisboa/LAMAR>. Acesso em: Jul. 2026.